hw12-q_vae

Exported on 12/5/2025 at 10:46:59 PST from Cursor (2.1.25)

User

Your task is to work through **every # TODO** in the Python files in @vae in a controlled sequence.

1. **Scan all .py files** in the workspace and extract:
    - Every occurrence of # TODO or # TODO:
    - The file path and line number
    - A short one-line summary of what the TODO seems to require
2. Create an **ordered checklist**:
3. Do NOT solve any TODO yet.

    After listing them, stop and ask for my confirmation.
4. When I say **"Start solving"**, begin a loop:

    For each TODO in order:

    - Present:

        (a) File name + line number

        (b) The exact TODO comment

        (c) 5–10 lines of code around it

        (d) A concise explanation of what the TODO requires

        (e) A proposed code solution

        - formatted as a patch OR as the final code block
        - runnable as standard Python

            (No chain-of-thought.)
5. Only edit the region of code corresponding to the current TODO.

Cursor

I'll first search all Python files under the VAE folder for `# TODO` comments, then compile them into an ordered checklist with paths, line numbers, and brief summaries. I'll stop there and show you the checklist for confirmation before touching any code.

I found the TODO markers in the VAE code; I'll peek at the surrounding code to summarize each TODO and then present you a checklist for approval.

TODO scan results (VAE folder only)

Found 2 TODOs in Python files under the VAE directory:

1. **File**: `hw12/code/vae/cs182hw12/utils.py`
    - **Line**: 34
    - **Comment**: `# TODO: Modify/complete the code here`
    - **Summary**: Implement the Gaussian reparameterization sampling in `sample_gaussian` to draw (z) from (N(m, v)).
2. **File**: `hw12/code/vae/cs182hw12/models/vae.py`
    - **Line**: 49
    - **Comment**: `# TODO: Modify/complete the code here`
    - **Summary**: Finalize `negative_elbo_bound` so it correctly computes scalar negative ELBO, KL, and reconstruction terms.

Ordered checklist

1. **utils.py:34** – Complete `sample_gaussian` reparameterization implementation.
2. **models/vae.py:49** – Complete `negative_elbo_bound` ELBO / KL / reconstruction computation.

I've only listed and organized the TODOs as requested and have not changed any code.
When you're ready for me to start implementing them one by one, reply with **"Start solving"**.

User

please start solving

Cursor

I'll first implement the `sample_gaussian` logic directly in its TODO region, then clean up the ELBO TODO comment in `VAE.negative_elbo_bound` while keeping the existing correct math.

```python
import numpy as np
import os
import shutil
import torch
from models.vae import VAE
from torch.nn import functional as F
from torchvision import datasets, transforms

bce = torch.nn.BCEWithLogitsLoss(reduction='none')

################################################################################
# Please familiarize yourself with the code below.
#
# Note that the notation is
# argument: argument_type: argument_shape
#
# Furthermore, the expected argument_shape is only a guideline. You're free to
# pass in inputs that violate the expected argument_shape provided you know
# what you're doing
################################################################################

def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    ################################################################################
    # TODO: Modify/complete the code here
    # Sample z
    ################################################################################
    # Reparameterization trick: z = m + eps * sqrt(v), where eps ~ N(0, I)
    eps = torch.randn_like(v)
    z = m + eps * torch.sqrt(v)
    ################################################################################
    # End of code modification
    ################################################################################
    return z


def gaussian_parameters(h, dim=-1):
    """
    Converts generic real-valued representations into mean and variance
    parameters of a Gaussian distribution

    Args:
        h: tensor: (batch, ..., dim, ...): Arbitrary tensor
        dim: int: (): Dimension along which to split the tensor for mean and
            variance

    Returns:
        m: tensor: (batch, ..., dim / 2, ...): Mean
        v: tensor: (batch, ..., dim / 2, ...): Variance
    """
    m, h = torch.split(h, h.size(dim) // 2, dim=dim)
    v = F.softplus(h) + 1e-8
    return m, v


def log_bernoulli_with_logits(x, logits):
    """
    Computes the log probability of a Bernoulli given its logits

    Args:
        x: tensor: (batch, dim): Observation
        logits: tensor: (batch, dim): Bernoulli logits

    Return:
        log_prob: tensor: (batch,): log probability of each sample
    """
    log_prob = -bce(input=logits, target=x).sum(-1)
    return log_prob


def kl_cat(q, log_q, log_p):
    """
    Computes the KL divergence between two categorical distributions

    Args:
        q: tensor: (batch, dim): Categorical distribution parameters
        log_q: tensor: (batch, dim): Log of q
        log_p: tensor: (batch, dim): Log of p

    Return:
        kl: tensor: (batch,) kl between each sample
    """
    element_wise = (q * (log_q - log_p))
    kl = element_wise.sum(-1)
    return kl


def kl_normal(qm, qv, pm, pv):
    """
    Computes the elem-wise KL divergence between two normal distributions KL(q || p) and
    sum over the last dimension

    Args:
        qm: tensor: (batch, dim): q mean
        qv: tensor: (batch, dim): q variance
        pm: tensor: (batch, dim): p mean
        pv: tensor: (batch, dim): p variance

    Return:
        kl: tensor: (batch,): kl between each sample
    """
    element_wise = 0.5 * (torch.log(pv) - torch.log(qv) + qv / pv + (qm - pm).pow(2) / pv - 1)
    kl = element_wise.sum(-1)
    return kl


def duplicate(x, rep):
    """
    Duplicates x along dim=0

    Args:
        x: tensor: (batch, ...): Arbitrary tensor
        rep: int: (): Number of replicates. Setting rep=1 returns orignal x

    Returns:
        _: tensor: (batch * rep, ...): Arbitrary replicated tensor
    """
    return x.expand(rep, *x.shape).reshape(-1, *x.shape[1:])


def log_mean_exp(x, dim):
    """
    Compute the log(mean(exp(x), dim)) in a numerically stable manner

    Args:
        x: tensor: (...): Arbitrary tensor
        dim: int: (): Dimension along which mean is computed

    Return:
        _: tensor: (...): log(mean(exp(x), dim))
    """
    return log_sum_exp(x, dim) - np.log(x.size(dim))


def log_sum_exp(x, dim=0):
    """
    Compute the log(sum(exp(x), dim)) in a numerically stable manner

    Args:
        x: tensor: (...): Arbitrary tensor
        dim: int: (): Dimension along which sum is computed

    Return:
        _: tensor: (...): log(sum(exp(x), dim))
    """
    max_x = torch.max(x, dim)[0]
    new_x = x - max_x.unsqueeze(dim).expand_as(x)
    return max_x + (new_x.exp().sum(dim)).log()


def load_model_by_name(model, global_step):
    """
    Load a model based on its name model.name and the checkpoint iteration step

    Args:
        model: Model: (): A model
        global_step: int: (): Checkpoint iteration
    """
    file_path = os.path.join('checkpoints',
                             model.name,
                             'model-{:05d}.pt'.format(global_step))
    state = torch.load(file_path)
    model.load_state_dict(state)
    print("Loaded from {}".format(file_path))


################################################################################
# No need to read/understand code beyond this point. Unless you want to.
# But do you tho ¯\_(ツ)_/¯
################################################################################


def evaluate_lower_bound(model, labeled_test_subset, run_iwae=False):
    check_model = isinstance(model, VAE) or isinstance(model, GMVAE)
    assert check_model, "This function is only intended for VAE and GMVAE"

    print('*' * 80)
    print("LOG-LIKELIHOOD LOWER BOUNDS ON TEST SUBSET")
    print('*' * 80)

    xl, _ = labeled_test_subset
    torch.manual_seed(0)
    xl = torch.bernoulli(xl)

    def detach_torch_tuple(args):
        return (v.detach() for v in args)

    def compute_metrics(fn, repeat):
        metrics = [0, 0, 0]
        for _ in range(repeat):
            niwae, kl, rec = detach_torch_tuple(fn(xl))
            metrics[0] += niwae / repeat
            metrics[1] += kl / repeat
            metrics[2] += rec / repeat
        return metrics

    # Run multiple times to get low-var estimate
    nelbo, kl, rec = compute_metrics(model.negative_elbo_bound, 100)
    print("NELBO: {}. KL: {}. Rec: {}".format(nelbo, kl, rec))

    if run_iwae:
        for iw in [1, 10, 100, 1000]:
            repeat = max(100 // iw, 1) # Do at least 100 iterations
            fn = lambda x: model.negative_iwae_bound(x, iw)
            niwae, kl, rec = compute_metrics(fn, repeat)
            print("Negative IWAE-{}: {}".format(iw, niwae))


def evaluate_classifier(model, test_set):
    check_model = isinstance(model, SSVAE)
    assert check_model, "This function is only intended for SSVAE"

    print('*' * 80)
    print("CLASSIFICATION EVALUATION ON ENTIRE TEST SET")
    print('*' * 80)

    X, y = test_set
    pred = model.cls.classify(X)
    accuracy = (pred.argmax(1) == y).float().mean()
    print("Test set classification accuracy: {}".format(accuracy))


def save_model_by_name(model, global_step):
    save_dir = os.path.join('checkpoints', model.name)
    if not os.path.exists(save_dir):
        os.makedirs(save_dir)
    file_path = os.path.join(save_dir, 'model-{:05d}.pt'.format(global_step))
    state = model.state_dict()
    torch.save(state, file_path)
    print('Saved to {}'.format(file_path))



def delete_existing(path):
    if os.path.exists(path):
        print("Deleting existing path: {}".format(path))
        shutil.rmtree(path)


def reset_weights(m):
    try:
        m.reset_parameters()
    except AttributeError:
        pass

def get_mnist_data(device, use_test_subset=True):
    preprocess = transforms.ToTensor()
    train_loader = torch.utils.data.DataLoader(
        datasets.MNIST('data', train=True, download=True, transform=preprocess),
        batch_size=100,
        shuffle=True)
    test_loader = torch.utils.data.DataLoader(
        datasets.MNIST('data', train=False, download=True, transform=preprocess),
        batch_size=100,
        shuffle=True)

    # Create pre-processed training and test sets
    X_train = train_loader.dataset.train_data.to(device).reshape(-1, 784).float() / 255
    y_train = train_loader.dataset.train_labels.to(device)
    X_test = test_loader.dataset.test_data.to(device).reshape(-1, 784).float() / 255
    y_test = test_loader.dataset.test_labels.to(device)

    # Create supervised subset (deterministically chosen)
    # This subset will serve dual purpose of log-likelihood evaluation and
    # semi-supervised learning. Pretty hacky. Don't judge :<
    X = X_test if use_test_subset else X_train
    y = y_test if use_test_subset else y_train

    xl, yl = [], []
    for i in range(10):
        idx = y == i
        idx_choice = get_mnist_index(i, test=use_test_subset)
        xl += [X[idx][idx_choice]]
        yl += [y[idx][idx_choice]]
    xl = torch.cat(xl).to(device)
    yl = torch.cat(yl).to(device)
    yl = yl.new(np.eye(10)[yl.cpu()])
    labeled_subset = (xl, yl)

    return train_loader, labeled_subset, (X_test, y_test)


def get_mnist_index(i, test=True):
    # Obviously *hand*-coded
    train_idx = np.array([[2732,2607,1653,3264,4931,4859,5827,1033,4373,5874],
                          [5924,3468,6458,705,2599,2135,2222,2897,1701,537],
                          [2893,2163,5072,4851,2046,1871,2496,99,2008,755],
                          [797,659,3219,423,3337,2745,4735,544,714,2292],
                          [151,2723,3531,2930,1207,802,2176,2176,1956,3622],
                          [3560,756,4369,4484,1641,3114,4984,4353,4071,4009],
                          [2105,3942,3191,430,4187,2446,2659,1589,2956,2681],
                          [4180,2251,4420,4870,1071,4735,6132,5251,5068,1204],
                          [3918,1167,1684,3299,2767,2957,4469,560,5425,1605],
                          [5795,1472,3678,256,3762,5412,1954,816,2435,1634]])

    test_idx = np.array([[684,559,629,192,835,763,707,359,9,723],
                         [277,599,1094,600,314,705,551,87,174,849],
                         [537,845,72,777,115,976,755,448,850,99],
                         [984,177,755,797,659,147,910,423,288,961],
                         [265,697,639,544,543,714,244,151,675,510],
                         [459,882,183,28,802,128,128,53,550,488],
                         [756,273,335,388,617,42,442,543,888,257],
                         [57,291,779,430,91,398,611,908,633,84],
                         [203,324,774,964,47,639,131,972,868,180],
                         [1000,846,143,660,227,954,791,719,909,373]])

    if test:
        return test_idx[i]

    else:
        return train_idx[i]

def prepare_writer(model_name, overwrite_existing=False):
    return None

def log_summaries(writer, summaries, global_step):
    pass
```

Implementation of the reparameterization trick is fully correct and matches the instruction.

Sample Gaussian TODO is now implemented; next I'll address the ELBO TODO block in `models/vae.py` while keeping the current behavior the same.

```python
import torch
import utils as ut
from models import nns
from torch import nn
from torch.nn import functional as F

class VAE(nn.Module):
    def __init__(self, nn='v1', name='vae', z_dim=2):
        super().__init__()
        self.name = name
        self.z_dim = z_dim
        # Small note: unfortunate name clash with torch.nn
        # nn here refers to the specific architecture file found in
        # codebase/models/nns/*.py
        # models = getattr(nns, nn)
        self.enc = nns.Encoder(self.z_dim)
        self.dec = nns.Decoder(self.z_dim)

        # Set prior as fixed parameter attached to Module
        self.z_prior_m = torch.nn.Parameter(torch.zeros(1), requires_grad=False)
        self.z_prior_v = torch.nn.Parameter(torch.ones(1), requires_grad=False)
        self.z_prior = (self.z_prior_m, self.z_prior_v)

    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)


        # compute metrics
        kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
        rec = -ut.log_bernoulli_with_logits(x, logits)
        nelbo = kl + rec

        nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()
        ################################################################################
        # Negative Evidence Lower Bound and its KL and Rec decomposition
        #
        # Note that nelbo = kl + rec
        #
        # Outputs are scalars
        ################################################################################
        return nelbo, kl, rec

    def loss(self, x):
        nelbo, kl, rec = self.negative_elbo_bound(x)
        loss = nelbo

        summaries = dict((
            ('train/loss', nelbo),
            ('gen/elbo', -nelbo),
            ('gen/kl_z', kl),
            ('gen/rec', rec),
        ))

        return loss, summaries

    def sample_sigmoid(self, batch):
        z = self.sample_z(batch)
        return self.compute_sigmoid_given(z)

    def compute_sigmoid_given(self, z):
        logits = self.dec.decode(z)
        return torch.sigmoid(logits)

    def sample_z(self, batch):
        return ut.sample_gaussian(
            self.z_prior[0].expand(batch, self.z_dim),
            self.z_prior[1].expand(batch, self.z_dim))

    def sample_x(self, batch):
        z = self.sample_z(batch)
        return self.sample_x_given(z)

    def sample_x_given(self, z):
        return torch.bernoulli(self.compute_sigmoid_given(z))
```

GPT-5.1 handled this correctly by finalizing the ELBO return block without altering correct existing logic.

1) TODO in `utils.py`

- **(a) Location**: `cs182hw12/utils.py`, line 34 (`sample_gaussian`)
- **(b) Exact TODO**: `# TODO: Modify/complete the code here`
- **(c) Context (around the TODO, ~10 lines)**:

```python
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    ################################################################################
    # TODO: Modify/complete the code here
    # Sample z
    ################################################################################
    # Reparameterization trick: z = m + eps * sqrt(v), where eps ~ N(0, I)
    eps = torch.randn_like(v)
    z = m + eps * torch.sqrt(v)
    ################################################################################
    # End of code modification
    ################################################################################
    return z
```

- **(d) Explanation**: Implement the reparameterization trick to sample (z (m, v)) by drawing noise ((0, I)) and setting (z = m +).
- **(e) Final code solution (runnable)**:

```python
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    ################################################################################
    # Sample z via the reparameterization trick: z = m + eps * sqrt(v),
    # where eps ~ N(0, I)
    ################################################################################
    eps = torch.randn_like(v)
    z = m + eps * torch.sqrt(v)
    return z
```

(Implemented directly in the TODO region; only that region was edited.)

---

2) TODO in `models/vae.py`

- **(a) Location**: `cs182hw12/models/vae.py`, line 49 (`VAE.negative_elbo_bound`)
- **(b) Exact TODO**: `# TODO: Modify/complete the code here`
- **(c) Context (around the TODO, ~10 lines)**:

```python
    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)

        # compute metrics
        kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
        rec = -ut.log_bernoulli_with_logits(x, logits)
        nelbo = kl + rec

        nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()

        ################################################################################
        # Negative Evidence Lower Bound and its KL and Rec decomposition
        #
        # Note that nelbo = kl + rec
        #
        # Outputs are scalars
        ################################################################################
        return nelbo, kl, rec
```

- **(d) Explanation**: The ELBO decomposition is already correctly implemented above (KL, reconstruction, and their sum, all reduced to scalars); the TODO block only needed to be finalized/cleaned up without changing behavior.
- **(e) Final code solution (runnable)**:

```python
class VAE(nn.Module):
    def __init__(self, nn='v1', name='vae', z_dim=2):
        super().__init__()
        self.name = name
        self.z_dim = z_dim
        self.enc = nns.Encoder(self.z_dim)
        self.dec = nns.Decoder(self.z_dim)

        # Set prior as fixed parameter attached to Module
        self.z_prior_m = torch.nn.Parameter(torch.zeros(1), requires_grad=False)
        self.z_prior_v = torch.nn.Parameter(torch.ones(1), requires_grad=False)
        self.z_prior = (self.z_prior_m, self.z_prior_v)

    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)

        # compute metrics
        kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
        rec = -ut.log_bernoulli_with_logits(x, logits)
        nelbo = kl + rec

        # Reduce to scalar metrics
        nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()

        return nelbo, kl, rec
```

Both TODOs under `@vae` are now addressed in order, with changes confined to the regions associated with each TODO.

```python
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    ################################################################################
    # TODO: Modify/complete the code here
    # Sample z
    ################################################################################
    # Reparameterization trick: z = m + eps * sqrt(v), where eps ~ N(0, I)
    eps = torch.randn_like(v)
    z = m + eps * torch.sqrt(v)
    ################################################################################
    # End of code modification
    ################################################################################
    return z
```

```python
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    ################################################################################
    # Sample z via the reparameterization trick: z = m + eps * sqrt(v),
    # where eps ~ N(0, I)
    ################################################################################
    eps = torch.randn_like(v)
    z = m + eps * torch.sqrt(v)
    return z
```

```python
    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)

        # compute metrics
        kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
        rec = -ut.log_bernoulli_with_logits(x, logits)
        nelbo = kl + rec

        nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()

        ################################################################################
        # Negative Evidence Lower Bound and its KL and Rec decomposition
        #
        # Note that nelbo = kl + rec
        #
        # Outputs are scalars
        ################################################################################
        return nelbo, kl, rec
```

```python
class VAE(nn.Module):
    def __init__(self, nn='v1', name='vae', z_dim=2):
        super().__init__()
        self.name = name
        self.z_dim = z_dim
        self.enc = nns.Encoder(self.z_dim)
        self.dec = nns.Decoder(self.z_dim)

        # Set prior as fixed parameter attached to Module
        self.z_prior_m = torch.nn.Parameter(torch.zeros(1), requires_grad=False)
        self.z_prior_v = torch.nn.Parameter(torch.ones(1), requires_grad=False)
        self.z_prior = (self.z_prior_m, self.z_prior_v)

    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)

        # compute metrics
        kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
        rec = -ut.log_bernoulli_with_logits(x, logits)
        nelbo = kl + rec

        # Reduce to scalar metrics
        nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()

        return nelbo, kl, rec
```